**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ**
**Ордена Трудового Красного Знамени**
**Федеральное государственное бюджетное образовательное учреждение высшего образования**
**«Московский технический университет связи и информатики»**

Кафедра «Программная инженерия»

# Лабораторная работа №4

по дисциплине «Структуры и алгоритмы обработки данных»
по теме «Минимальное соединение и эвристический поиск пути»

Выполнил студент группы:
БПИ2401
Еланская Екатерина Дмитриевна
Проверил: Борзов В.М.

Москва, 2026

# Оглавление

**Введение** . . . . . **3**

**Выполнение** . . . . . **4**
Исходные данные . . . . . 4
Задание 1. DSU и минимальное остовное дерево . . . . . 4
Задание 2. Пространство состояний и базовый BFS . . . . . 5
Задание 3. Эвристические функции . . . . . 7
Задание 4. Greedy Best-First Search . . . . . 7
Задание 5. Алгоритм A* . . . . . 8
Задание 6. Итоговое сравнение BFS, Дейкстры, Greedy и A* . . 9

**Вывод** . . . . . **12**

**Контрольные вопросы** . . . . . **13**

# Введение

Цель данной лабораторной работы:

— познакомиться с применением структуры непересекающихся множеств и алгоритма Краскала для построения минимального остовного дерева;

— изучить эвристические методы поиска пути, реализовать Greedy Best-First Search и A*;

— сравнить BFS, алгоритм Дейкстры, Greedy и A* по стоимости найденного маршрута и количеству обработанных состояний.

Задачи:

1) научиться различать задачу минимального остовного дерева и задачу кратчайшего пути;

2) научиться использовать DSU с оптимизациями, задавать пространство состояний на клеточном поле;

3) научиться рассчитывать эвристические оценки;

4) научиться организовывать OPEN и CLOSED;

5) научиться восстанавливать маршрут и экспериментально оценивать влияние эвристики на эффективность поиска.

# Выполнение

## Исходные данные

Исходные граф для задания 1 и клеточное поле для заданий 2-6 представлены в листинге 1.

```
V = { 'A', 'B', 'C', 'D', 'E', 'F', 'G' }
edges = [
    (0, 1, 4),
    (0, 2, 3),
    (1, 2, 1),
    (1, 3, 2),
    (2, 3, 4),
    (2, 4, 2),
    (3, 4, 3),
    (3, 5, 2),
    (4, 5, 3),
    (4, 6, 6),
    (5, 6, 1)
]

N = 12
# S - start, G - target, . - empty, # - obstacle
field = [
    'S..#........',
    '.#.#.######.',
    '.#...#......',
    '.###.#.####.',
    '.....#....#.',
    '.#####.##.#.',
    '.......##...',
    '.#####....#.',
    '.#...####.#.',
    '.#.#......#.',
    '...#.####...',
    '####......G.'
]
S = (0, 0)
G = (11, 10)
```

Листинг 1 — Исходные данные

## Задание 1. DSU и минимальное остовное дерево

Реализация DSU и алгоритма Краскала представлены в листинге 2. Результат работы представлен в листинге 3.

```
class DSU:
    def __init__(self, vertex_cnt):
        self.parent = list(range(vertex_cnt))
        self.size = [1] * vertex_cnt

    def find(self, v):
        if self.parent[v] != v:
            self.parent[v] = self.find(self.parent[v])
        return self.parent[v]

    def union(self, a, b):
```

```
        a = self.find(a)
        b = self.find(b)
        if a == b:
            return False
        if self.size[a] < self.size[b]:
            a, b = b, a
        self.parent[b] = a
        self.size[a] += self.size[b]
        return True

def kruskal(edges, vertex_cnt):
    dsu = DSU(vertex_cnt)
    mst = []
    total = 0
    for u, v, w in sorted(edges, key=lambda e: e[2]):
        if dsu.union(u, v):
            mst.append((u, v, w))
            total += w
            if len(mst) == vertex_cnt - 1:
                break
    return mst, total
```

Листинг 2 — Класс DSU и алгоритм Краскала

```
A - B : 4
A - C : 3
B - D : 2
C - D : 4
E - G : 6
F - G : 1
total_weight=21
```

Листинг 3 — Вывод минимального остовного дерева графа и суммы его весов

Кратчайший путь минимизирует стоимость маршрута между двумя выбранными вершинами, MST минимизирует суммарную стоимость соединения всех вершин. Таким образом, ребра MST не обязаны лежать на кратчайших путях от какой-либо вершины.

## Задание 2. Пространство состояний и базовый BFS

Реализация BFS представлена в листинге 4. Полученный путь представлен в листинге 5.

```
def get_neighbours(a):
    row, col = a
    res = []
    # down, up, right, left
    for delta_row, delta_col in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        new_row = row + delta_row
        new_col = col + delta_col
        if 0 <= new_row < N and 0 <= new_col < N and
           field[new_row][new_col] != '#':
            res.append((new_row, new_col))
    return res

def reconstruct_path(parent, goal):
```

```
    path = []
    node = goal
    while node is not None:
        path.append(node)
        node = parent[node]
    path.reverse()
    return path

def bfs(S, G, cell_cost=None):
    queue = deque([S])
    parent = {S: None}
    dist = {S: 0}
    processed = 0
    while queue:
        cur = queue.popleft()
        processed += 1
        if cur == G:
            break
        for neighbour in get_neighbours(cur):
            if neighbour  not in parent:
                parent[neighbour] = cur
                if cell_cost is None:
                    cur_cost = 1
                else:
                    cur_cost = cell_cost(neighbour)
                dist[neighbour] = dist[cur] + cur_cost
                queue.append(neighbour)

    if G not in parent:
        return None, None, processed
    return reconstruct_path(parent, G), dist[G], processed

def render(path):
    grid = [list(row) for row in field]
    for (row, col) in path:
        if grid[row][col] == '.':
            grid[row][col] = '*'
    return '\n'.join(''.join(row) for row in grid)
```

Листинг 4 — Алгорим поиска BFS

```
steps=21, processed=83
S..#........
*#.#.######.
*#...#......
*###.#.####.
*....#....#.
*#####.##.#.
********##...
.##########.
.#...#####*#.
.#.#.....*#.
...#.####*..
####.....*G.
```

Листинг 5 — Вывод пути по алгоритму BFS

## Задание 3. Эвристические функции

Реализация функций эвристики представлены в листинге 6. Результат их работы представлен в листинге 7.

```
def manhattan(a, b):
    h = abs(a[0] - b[0]) + abs(a[1] - b[1])
    return h

def euclidean(a, b):
    h = hypot(a[0] - b[0], a[1] - b[1])
    return h
```

Листинг 6 — Функции manhattan(a, b) и euclidean(a, b)

```
p=(0, 0): manhattan=21 euclidean=14.866
p=(6, 0): manhattan=15 euclidean=11.18
p=(6, 6): manhattan=9 euclidean=6.403
p=(7, 9): manhattan=5 euclidean=4.123
p=(11, 9): manhattan=1 euclidean=1.0
```

Листинг 7 — Вывод эвристик для клеток

При переходах только по четырем направлениям каждый шаг меняет ровно одну координату на 1, поэтому минимально необходимое число шагов точно равно манхэттенскому расстоянию. Евклидово расстояние – длина прямой, которую агент пройти не может (диагонали запрещены), поэтому оно систематически занижает оценку и дает менее направленный поиск.

При единичной стоимости шага реальная стоимость пути больше манхэттенского расстояния (каждый шаг уменьшает манхэттен до цели не более чем на 1). Евклидово меньше манхэттенского, значит тоже не завышает.

## Задание 4. Greedy Best-First Search

Реализация Greedy поиска представлены в листинге 8. Результат работы представлен в листинге 9.

```
def greedy(S, G, h=manhattan):
    open_heap = [(h(S, G), 0, S)]
    counter = 1
    parent = {S: None}
    opened = {S}   # OPEN
    closed = set() # CLOSED
    processed = 0
    while open_heap:
        _, _, cur = hp.heappop(open_heap)
        if cur in closed:
            continue
        opened.discard(cur)
        closed.add(cur)
        processed += 1
        if cur == G:
            break
        for neighbour in get_neighbours(cur):
```

```
            if neighbour in closed:
                continue
            if neighbour not in parent:
                parent[neighbour] = cur
                hp.heappush(open_heap, (
                        h(neighbour, G),
                        counter, neighbour
                    ))
                counter += 1
                opened.add(neighbour)

    if G not in parent:
        return None, None, processed
    path = reconstruct_path(parent, G)

    return path, len(path) - 1, processed
```

Листинг 8 — Алгорим поиска Greedy Best-First Search

```
steps=25 processed=26
S..#........
*#.#.######.
*#...#......
*###.#.####.
*....#....#.
*#####.##.#.
*......##...
*#####....#.
*#########.#.
*###*.....#.
****#####...
##########G.
```

Листинг 9 — Вывод пути по алгоритму Greedy

Greedy учитывает только h(n) (оценку расстояния до цели) и полностью игнорирует g(n) (стоимость уже пройденного пути). Он может пойти в ветку, которая выглядит близкой к цели, но ведет к длинному или дорогому маршруту, и не имеет механизма вернуться и пересмотреть выбор.

## Задание 5. Алгоритм A*

Реализация алгоритма A* представлены в листинге 10.

```
def aast(S, G, h=manhattan, cell_cost=None):
    g = {S: 0}
    f = {S: h(S, G)}
    parent = {S: None}
    open_heap = [(f[S], 0, S)]
    counter = 1
    closed = set()
    processed = 0

    while open_heap:
        _, _, cur = hp.heappop(open_heap)
        if cur in closed:
            continue
```

```
        closed.add(cur)
        processed += 1
        if cur == G:
            break
        for neighbour in get_neighbours(cur):
            if neighbour in closed:
                continue
            if cell_cost is None:
                cur_cost = 1
            else:
                cur_cost = cell_cost(neighbour)
            tent_dist = g[cur] + cur_cost
            if neighbour not in g or tent_dist < g[neighbour]:
                g[neighbour] = tent_dist
                f[neighbour] = tent_dist + h(neighbour, G)
                parent[neighbour] = cur
                hp.heappush(open_heap, (
                    f[neighbour],
                    counter,
                    neighbour
                ))
                counter += 1

    if G not in g:
        return None, None, processed
    return reconstruct_path(parent, G), g[G], processed
```

Листинг 10 — Алгорим поиска A*

```
manhatten: cost=21 processed=40
S..#........
*#.#.######.
*#...#......
*###.#.####.
*....#....#.
*#####.##.#.
********##...
.##########.
.#...#####*#.
.#.#.....*#.
...#.####*..
####.....*G.
euclidean: cost=21 processed=43
```

Листинг 11 — Вывод пути по алгоритму A*

На данном поле манхэттенская эвристика дала более направленный поиск, так как у нее меньше обработанных состояний при той же оптимальной длине.

## Задание 6. Итоговое сравнение BFS, Дейкстры, Greedy и A*

Реализация алгоритма Дейкстры представлена в листинге 12.

```
def dijkstra(S, G, cell_cost=None):
    dist = {S: 0}
    parent = {S: None}
```

```
    heap = [(0, 0, S)]
    counter = 1
    closed = set()
    processed = 0

    while heap:
        cur_dist, _, cur = hp.heappop(heap)
        if cur in closed:
            continue
        closed.add(cur)
        processed += 1
        if cur == G:
            break
        for neighbour in get_neighbours(cur):
            if neighbour in closed:
                continue
            if cell_cost is None:
                cur_cost = 1
            else:
                cur_cost = cell_cost(neighbour)
            new_dist = cur_dist + cur_cost
            if neighbour not in dist or new_dist < dist[neighbour]:
                dist[neighbour] = new_dist
                parent[neighbour] = cur
                hp.heappush(heap, (
                    new_dist,
                    counter,
                    neighbour
                ))
                counter += 1

    return reconstruct_path(parent, G), dist.get(G), processed
```

Листинг 12 — Алгорим Дейкстры

Итог сравнения алгоритмов представлен в таблице 1.

Таблица 1 — Сравнение алгоритмов поиска пути

| **Алгоритм** | **Стоимость маршрута** | **Длина маршрута** | **Обработано состояний** | **Гарантия оптимальности** |
|---|---|---|---|---|
| BFS | 41 | 21 | 83 | при одинаковых весах |
| Дейкстра | 25 | 25 | 78 | да |
| Greedy | 25 | 25 | 26 | нет |
| A* Manhattan | 25 | 25 | 54 | при допустимой эвристике |
| A* Euclidean | 25 | 25 | 51 | при допустимой эвристике |

Гарантия оптимальности (Дейкстра и A*) стоит обработки большего числа состояний. Greedy обрабатывает мало, но не гарантирует оптимальность. A* занимает промежуточное положение: при корректной эвристике сохраняет оптимальность и сокращает область поиска по сравнению с Дейкстрой. Манхэттенская оценка оптимальнее эвклидовой на

четырехсвязной сетке.

# Вывод

В ходе лабораторной работы были:

— получены знания о применении структуры непересекающихся множеств и алгоритма Краскала для построения минимального остовного дерева;

— изучены эвристические методы поиска пути, реализовать Greedy Best-First Search и A*;

— проведено сравнение BFS, алгоритм Дейкстры, Greedy и A* по стоимости найденного маршрута и количеству обработанных состояний.

# Контрольные вопросы

1) *Что называется остовным деревом графа?*
   Остовным деревом связного неориентированного графа называется подграф, содержащий все вершины исходного графа, остающийся связным и не содержащий циклов.

2) *Чем минимальное остовное дерево отличается от кратчайшего пути?*
   Кратчайший путь минимизирует стоимость маршрута между выбранными вершинами, а MST минимизирует суммарную стоимость соединения всех вершин.

3) *Для чего используется структура DSU?*
   Структура непересекающихся множеств DSU (Disjoint Set Union, Union-Find) поддерживает разбиение элементов на непересекающиеся компоненты.

4) *Что выполняют операции find и union?*
   Операция find(x) возвращает представителя компоненты, union(a, b) – объединяет две компоненты.

5) *Как работают сжатие путей и объединение по рангу или размеру?*
   Сжатие find(a) все вершины на пути от a до корня переподвешиваются напрямую к корню. Объединение union(a, b) объединяет две компоненты: если по рангу, то корень дерева с меньшим рангом (оценкой высоты) подвешивается к корню с большим рангом; если по размеру, то то же самое, но по количеству элементов в компонентах.

6) *В чём состоит жадный принцип алгоритма Краскала?*
   Алгоритм Краскала сортирует все рёбра по весу и последовательно добавляет самое легкое ребро, если оно соединяет две разные компоненты.

7) *Чем принцип работы Прима отличается от Краскала?*
   Алгоритм Прима строит остов от выбранной начальной вершины, каждый раз добавляя самое лёгкое ребро, которое соединяет уже включенную часть дерева с ещё не включённой вершиной.

8) *Что называется пространством состояний в задаче поиска пути?*
   Задачу поиска маршрута можно представить как поиск в пространстве состояний. Состоянием является положение агента, а ребра графа соответствуют допустимым переходам.

9) *Что такое эвристическая функция?*
Эвристическая функция h(n) – такая быстро вычисляемая функция. которая отражает направление к цели и оценивает оставшуюся стоимость пути от текущего состояния n до цели. Она не обязана совпадать с реальной стоимостью пути.

10) *Чем манхэттенское расстояние отличается от евклидова?*
Манхэттенская оценка обычно точнее отражает минимально необходимое число шагов, тогда как евклидова оценка учитывает прямое геометрическое расстояние.

11) *Для каких задач может использоваться расстояние Хэмминга?*
Расстояние Хэмминга подсчитывает количество позиций, в которых два состояния отличаются. Может применяться в теории кодирования (для обнаружения и исправления ошибок); при сравнении двоичных последовательностей и хешей; в задачах-головоломках, где состояние – перестановка, и число несовпадающих позиций служит эвристикой; в биоинформатике.

12) *Как Greedy Best-First Search выбирает следующую вершину?*
Он выбирает из множества OPEN состояние с минимальной эвристической оценкой h(n).

13) *Почему Greedy не гарантирует оптимальный маршрут?*
Алгоритм тремится быстрее приблизиться к цели, но не учитывает стоимость уже пройденного пути, поэтому часто рассматривает мало состояний.

14) *Что означают g(n), h(n) и f(n) в A*?*
g(n) – фактическая стоимость пути от старта до n; h(n) – оценка оставшейся стоимости; f(n) – суммарный приоритет состояния.

15) *Для чего в A* используются OPEN и CLOSED?*
OPEN хранит обнаруженные, но еще не обработанные состояния, обычно в приоритетной очереди. CLOSED содержит состояния, обработка которых завершена.

16) *Что означает допустимость эвристики?*
Эвристика называется допустимой, если она никогда не завышает истинную минимальную стоимость пути до цели.

17) *Что означает согласованность эвристики?*
Согласованная эвристика дополнительно к дпокстимости требует, чтобы оценка в текущей вершине не превышала стоимость перехода к соседу плюс оценку соседа.

18) *Во что превращается A* при h(n) = 0?*
В алгоритм Дейкстры.

19) *В каком случае BFS гарантирует оптимальный путь?*
BFS оптимален по числу ребер только при одинаковой стоимости переходов.

20) *Почему при взвешенных клетках минимальный по числу шагов маршрут может не быть минимальным по стоимости?*
Потому что BFS и подсчет шагов минимизируют количество переходов, а не сумму весов. Если клетки имеют разную стоимость прохода, короткий путь из 5 шагов по дорогим клеткам может стоить дороже, чем путь из 8 шагов по дешевым.

21) *Почему корректная эвристика может уменьшить число обработанных состояний по сравнению с Дейкстрой?*
Дейкстра при выборе следующей вершины смотрит только на g(n) – пройденную стоимость, поэтому расходится во все стороны равномерно. A* с корректной эвристикой использует f(n)=g(n)+h(n), где h(n) направляет поиск в сторону цели. Вершины, явно удаленные от цели, получают большой f(n) и обрабатываются позже или не обрабатываются вовсе. При этом допустимость гарантирует, что оптимальный путь не будет отсечен. В итоге A* исследует меньше состояний, сохраняя оптимальность.